MOSEF 2024-2025 12/2024

Projet

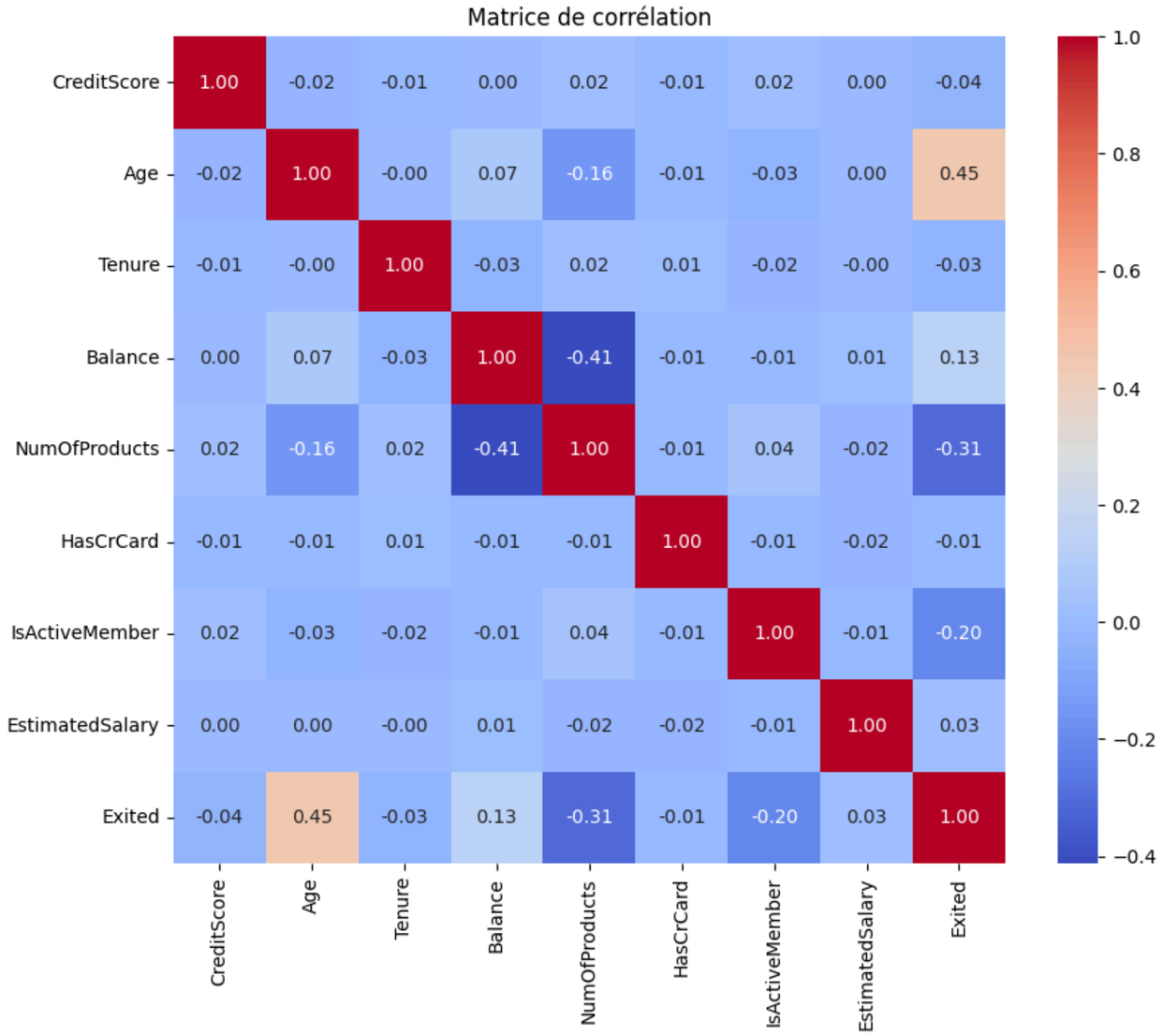
Data Mining

HUYNH Minh Nhat Thy
Sara Laval-Jeantet

Plan du projet

1. **Exploratory Data Analysis (EDA)**
2. **Quels modèles potentiels ?**
3. **Modélisation**
    a. **XGBClassifier**
        i. **XGB - Baseline**
        ii. **XGB - Feature Engineering**
        iii. **XGB - Tuning**
    b. **Stacking**
    c. **Voting**
    d. **Catboost**

1. EDA



Matrice de corrélation

- **Variables importantes :**
  - **Âge** : La corrélation la plus élevée avec le churn (ρ = 0.45), indiquant que les clients plus âgés ont tendance à churner davantage.
  - **Nombre de Produits** : Corrélation négative (ρ = -0.31) suggérant que les clients ayant plusieurs produits sont moins susceptibles de churner.
  - **Activité du membre** : Corrélation négative (ρ = -0.20), confirmant que l'activité des membres réduit le risque de churn.

- **Autres variables :**
  - Le solde (**Balance**) présente une corrélation faible (ρ = 0.13) avec le churn.
  - Le score de crédit (**CreditScore**) et le salaire estimé (**EstimatedSalary**) ont des corrélations négligeables (ρ ≈ 0).

- **Conclusion :**
  - L'âge, l'activité, et le nombre de produits sont des facteurs déterminants dans l'analyse du churn, tandis que d'autres variables comme le score de crédit jouent un rôle limité.

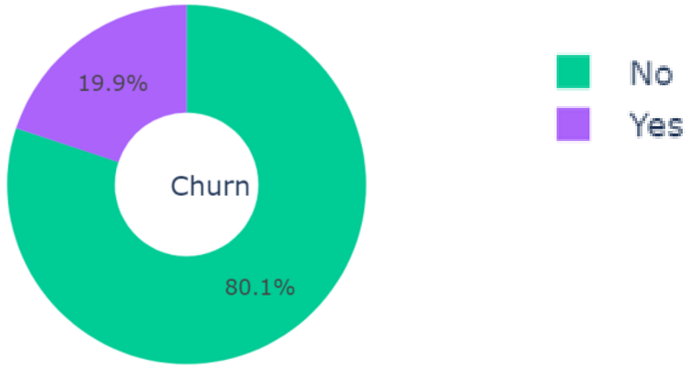
Introduction et Données

Description du jeu de données :

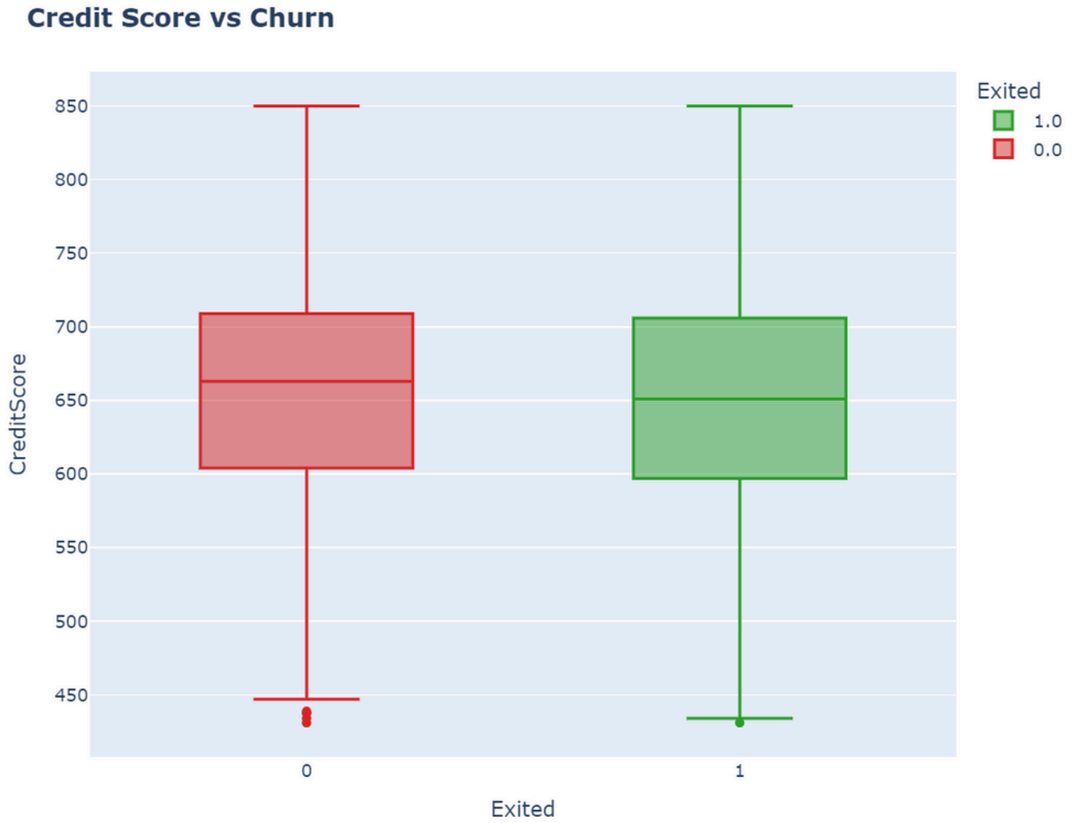
- Données client incluant démographie, score de crédit, ancienneté, activité de compte, et statut de churn.

Déséquilibre des classes :

- Le jeu de données présente un fort déséquilibre : environ **80 %** des clients n'ont pas churné (Exited = 0), contre seulement **20 %** qui ont churné (Exited = 1).

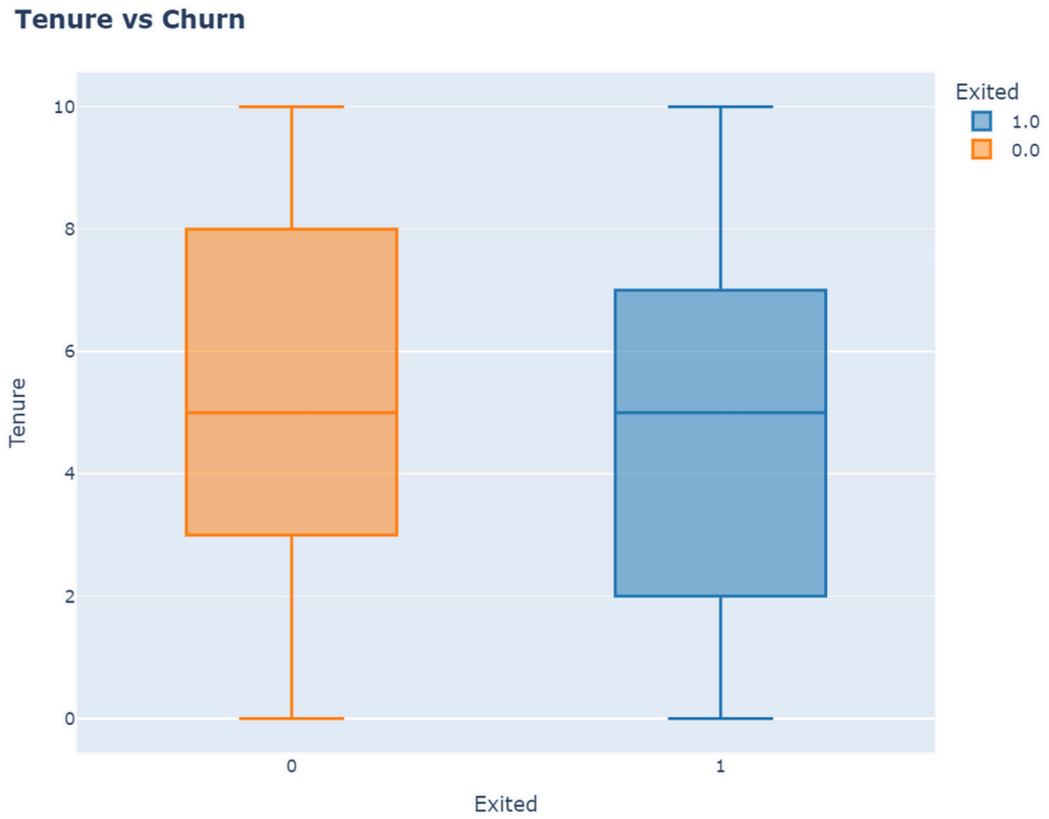


Analyse du Score de Crédit et de l'Ancienneté par Rapport au Churn



Score de crédit

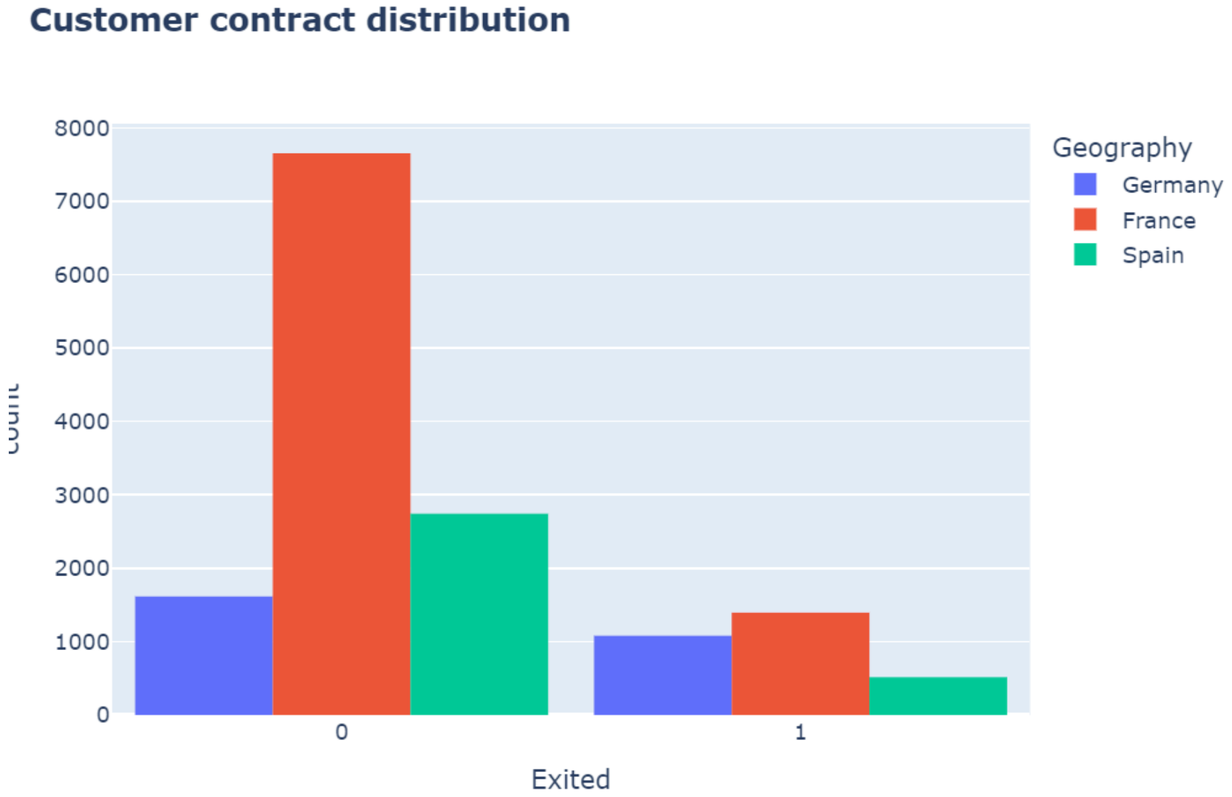
Les différences observées entre les groupes churn et non-churn sont faibles et pourraient ne pas être statistiquement significatives.



Ancienneté

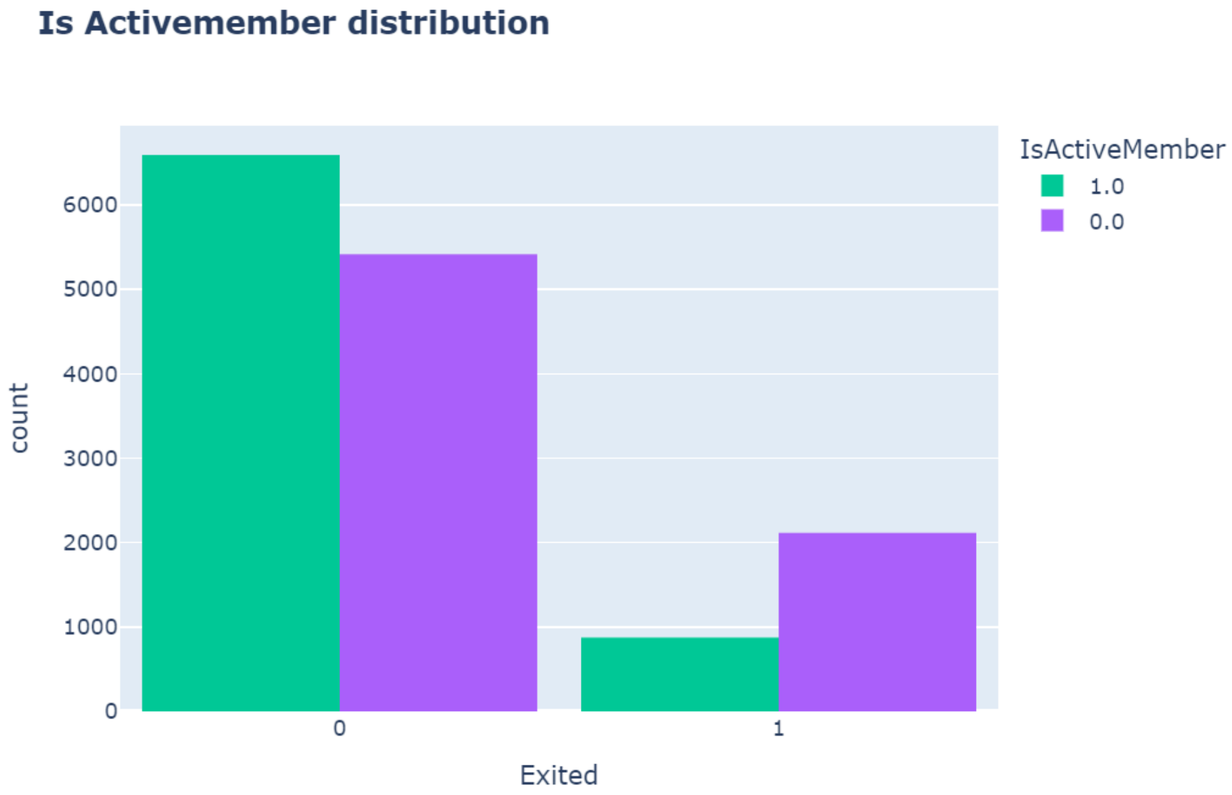
Les clients churnés tendent à avoir une ancienneté légèrement plus faible, avec une majorité ayant moins de 7 ans d’ancienneté, contre 8 ans pour les clients fidèles. Les différences restent modestes.

Impact de la Géographie et de l'Activité sur le Churn



Géographie

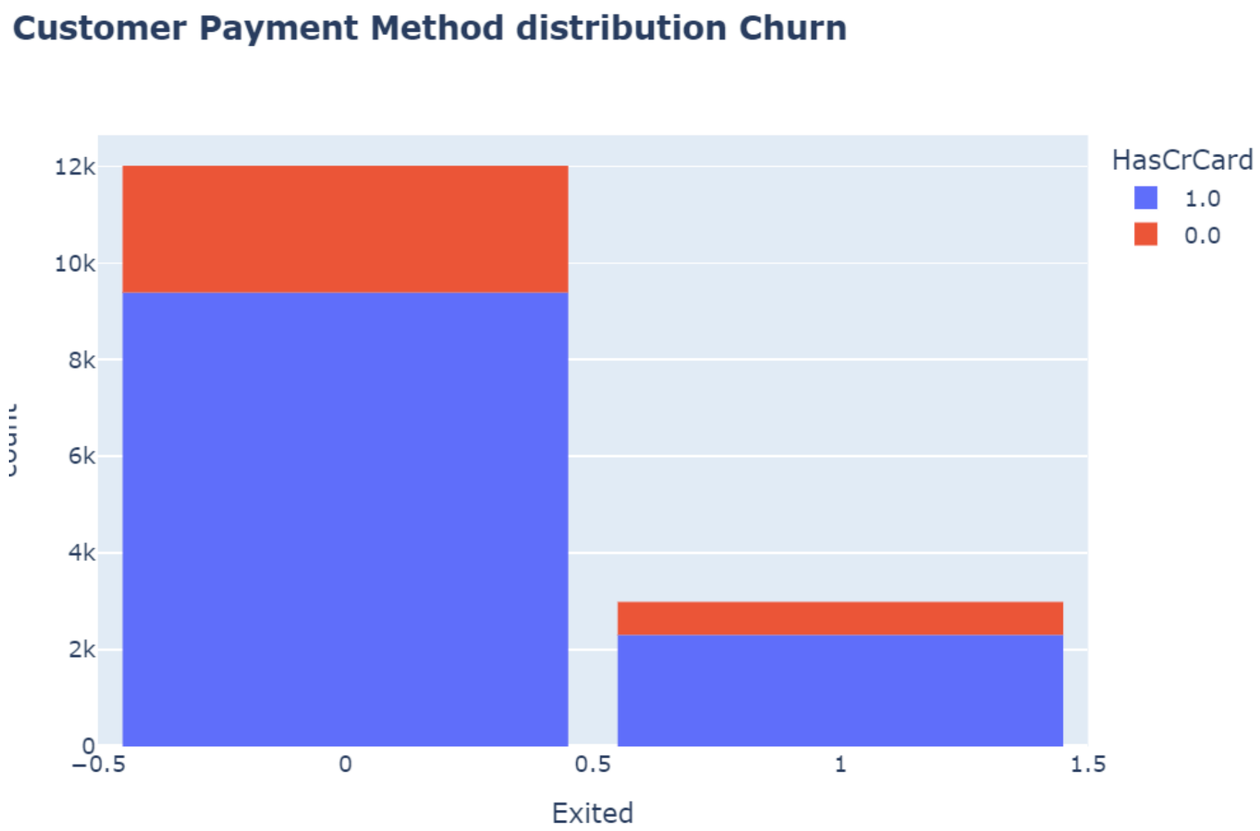
La France représente la plus grande proportion de clients avec un **faible taux de churn**.
L'Allemagne montre **un taux de churn proportionnellement plus élevé**, mais les différences globales entre pays ne sont pas fortement marquées



Activité des Membres

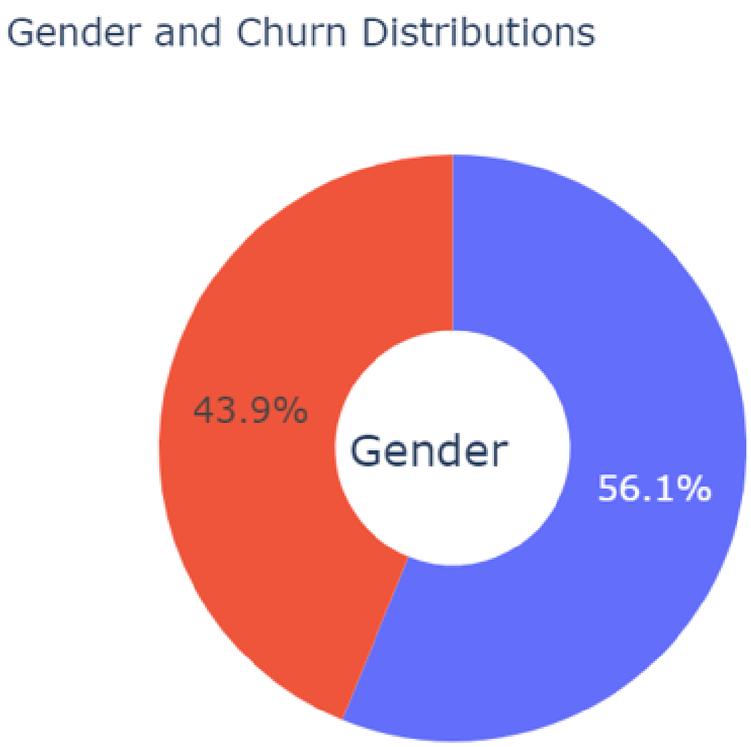
Les membres actifs sont **globalement moins susceptibles de churner**.

Influence des Moyens de Paiement et du Genre sur le Churn



Carte de Crédit

La possession d’une carte de crédit n’a pas d’impact notable sur le churn, la proportion dans les deux groupes était harmonieuse.



Genre

Les différences entre hommes et femmes en termes de churn sont faibles et probablement insignifiantes.

2. Quels modèles potentiels ?

Problème

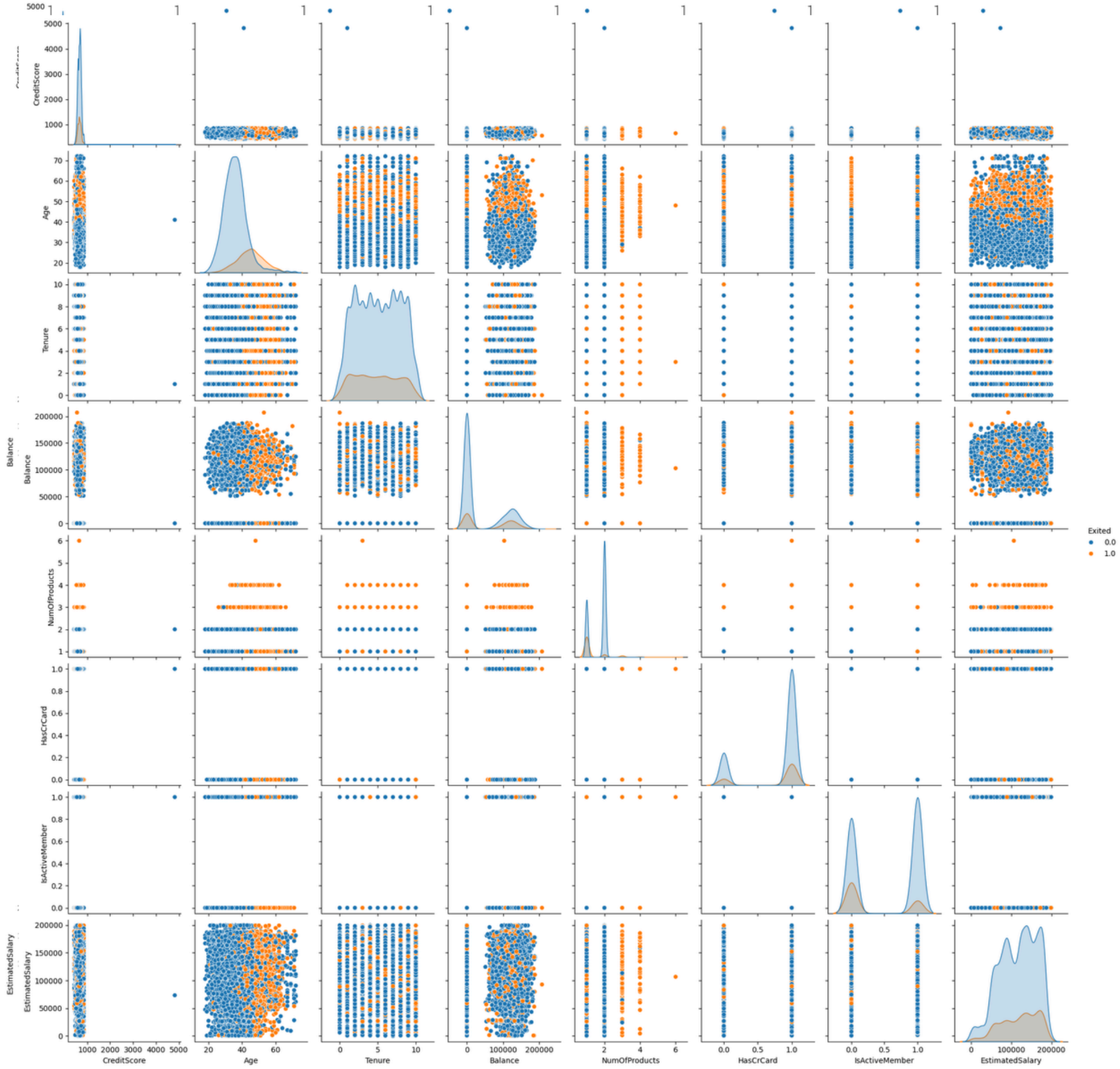
En pratique, le choix des modèles constitue l'une des tâches qui exige non seulement une expertise technique, mais aussi une solide connaissance du métier. Ainsi, déterminer quel modèle à adopter est un défi considérable. En général, il existe deux approches courantes pour rendre le choix du modèle plus clair.

- **Méthode 1** : Prédiction des modèles potentiels en se basant sur l'analyse du chevauchement (overlapping) à travers des pairplots

- **Méthode 2** : Prédiction des modèles potentiels en utilisant la bibliothèque LazyPredict (Ou toute autre bibliothèque, par exemple pycaret, etc. offrant la possibilité d'explorer des modèles de base (baselines)

(Lien : https://lazypredict.readthedocs.io/en/latest/)





Pairplot

En observant le pairplot, nous pouvons cibler certains modèles comme suit en analysant le niveau d'overlapping.

1. Support Vector Machines (SVM) :

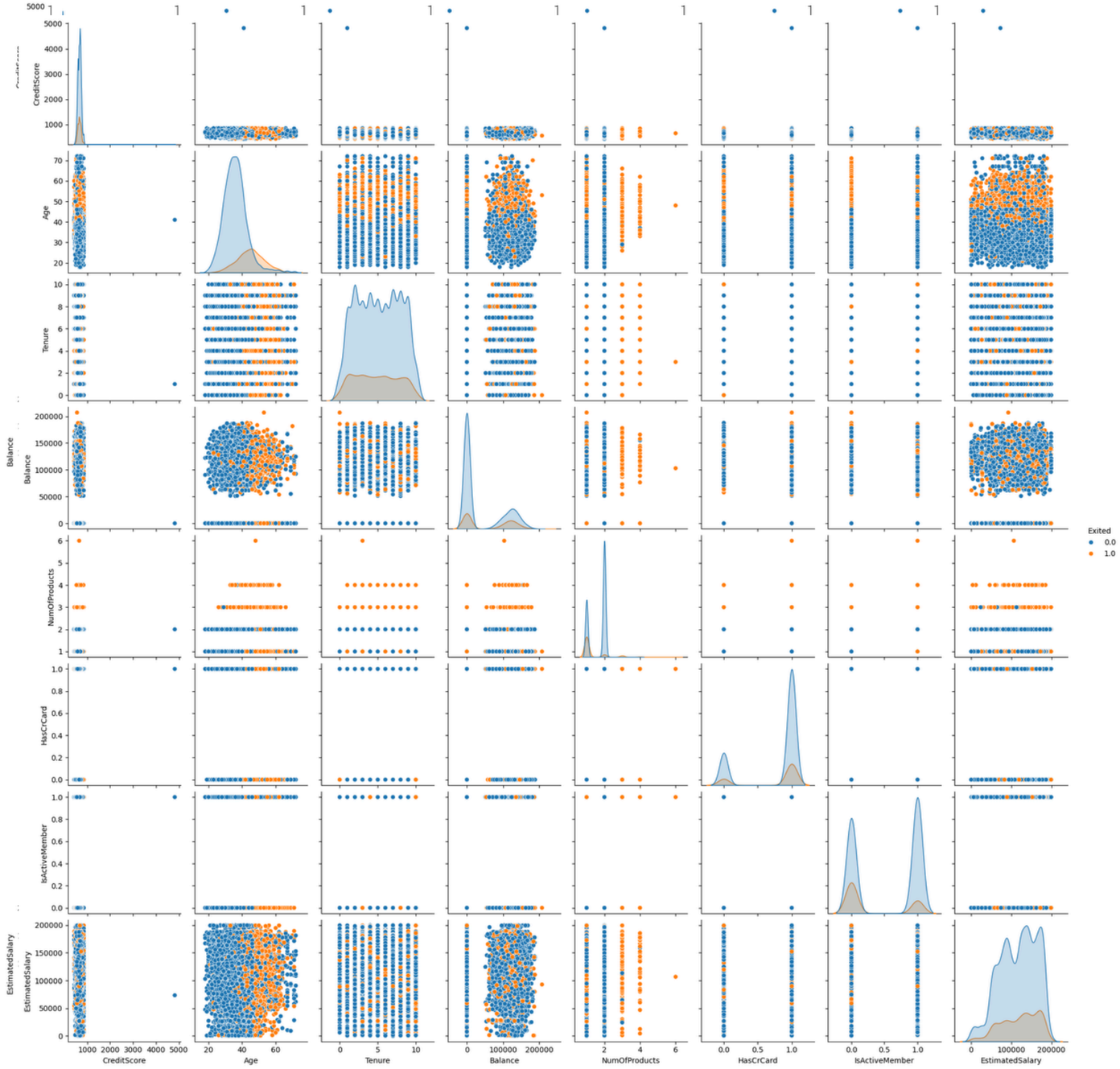
- Argument : Les niveaux de superposition faibles ou moyens pour certaines variables (par exemple, Age, Balance, et NumOfProducts) suggèrent que des frontières non linéaires peuvent bien fonctionner pour séparer les données. Un SVM avec un noyau non linéaire (par exemple, RBF) pourrait trouver des frontières optimales.

2. Arbres de décision (Decision Trees) :

- Argument : Certaines variables comme Age, IsActiveMember, et NumOfProducts sont bien discriminantes (faible superposition). Un arbre de décision peut facilement utiliser ces caractéristiques pour segmenter les données selon des seuils.

3. Forêts aléatoires (Random Forest) :

- Argument : Lorsque les caractéristiques présentent des niveaux de superposition variés (faibles à moyens), une forêt aléatoire peut combiner plusieurs arbres pour créer un modèle robuste, réduisant le surajustement. entraîner des erreurs accrues.



Pairplot

4. Gradient Boosting (GBM) :

- Argument : Les algorithmes de boosting (XGBoost, LightGBM, ou CatBoost) peuvent traiter efficacement des superpositions moyennes. Ils optimisent les frontières de classification en se basant sur les erreurs des modèles précédents.

5. Régression logistique (limite) :

- Argument : Bien que la régression logistique soit un modèle linéaire, elle pourrait fonctionner si des transformations des caractéristiques ou des variables d'interaction sont introduites pour capturer les relations non linéaires.

6. K-Nearest Neighbors (KNN) (non recommandé) :

- Argument : Les niveaux élevés de superposition dans certaines variables (comme CreditScore ou EstimatedSalary) rendent KNN moins performant, car il repose sur les distances. Cela pourrait entraîner des erreurs accrues.

Analyse les résultats par LazyPredict

```
Average ROC AUC scores across folds:
Model
LGBMClassifier                     0.81
AdaBoostClassifier                 0.81
XGBClassifier                      0.80
QuadraticDiscriminantAnalysis      0.80
RandomForestClassifier             0.80
SVC                                0.80
ExtraTreesClassifier               0.79
NearestCentroid                    0.79
BaggingClassifier                  0.78
KNeighborsClassifier               0.77
DecisionTreeClassifier             0.76
LinearDiscriminantAnalysis         0.76
LabelSpreading                     0.76
LabelPropagation                   0.76
LogisticRegression                 0.75
CalibratedClassifierCV             0.75
GaussianNB                         0.75
SGDClassifier                      0.74
LinearSVC                          0.74
BernoulliNB                        0.73
ExtraTreeClassifier                0.73
RidgeClassifier                    0.72
RidgeClassifierCV                  0.72
PassiveAggressiveClassifier        0.71
Perceptron                         0.70
DummyClassifier                    0.50
dtype: float64
```

Attention avec LazyPredict :

- Elle adopte une approche extrêmement **simplifiée** et **minimale** pour le traitement des données, dont le seul but est de permettre au modèle de s'exécuter (sans véritable optimisation des étapes de preprocessing comme nous le faisons manuellement).
- Nous savons que différents types de prétraitement (preprocessing) conduisent à des résultats différents. Par conséquent, **l'idée est d'utiliser les résultats de LazyPredict comme une base de comparaison "par défaut" (Stratified 5 Folds)**, puis de voir comment l'optimisation manuelle des étapes de preprocessing avec différentes techniques impacte les métriques de performance.

Analyse les résultats :

- Groupe **Boosting (LightGBM, AdaBoost, XGBoost)** offre les meilleures performances (ROC AUC : 0.80–0.81) et est le plus adapté à l’utilisation sur cet ensemble de données.
- Groupe **Bagging (Random Forest, Extra Trees, BaggingClassifier)** est également très performant, avec des résultats proches de ceux du Boosting.
- Les modèles linéaires tels que **Logistic Regression** et **RidgeClassifier** montrent des performances moyennes, indiquant que les données pourraient ne pas être entièrement adaptées aux modèles linéaires.

Donc, la suite des diapositives se concentrera sur le groupe des modèles Boosting ainsi que sur les techniques de Tuning, Stacking et Voting.

3. Modélisation

Preprocessing

```python
# Séparer les colonnes catégoriques et numériques
numerical_features = ['CreditScore', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard', 'IsActiveMember', 'EstimatedSalary']
categorical_features = ['Geography', 'Gender']
target_encoding_feature = ['Surname']

# Initialiser Target Encoder pour la colonne 'Surname'
target_encoder = ce.TargetEncoder(cols=target_encoding_feature)

# Pipeline pour traiter les colonnes catégoriques et numériques, incluant le Target Encoding pour 'Surname'
preprocessor = ColumnTransformer(
    transformers=[
        ('target_enc', target_encoder, target_encoding_feature),  # Appliquer le Target Encoding sur 'Surname'
        ('num', StandardScaler(), numerical_features),  # Traiter les colonnes numériques
        ('cat', OneHotEncoder(drop='first'), categorical_features)  # Traiter les colonnes catégoriques
    ])
```

Il s'agit d'un dataset **sans valeurs manquantes ni aberrations significatives** à traiter, ce qui signifie qu'aucune technique d'imputation particulière n'a été appliquée. Pour le prétraitement des données, j'ai adopté :

- **StandardScaler** pour les **numerical_features**,
- **OneHotEncoder** pour les **categorical_features**,
- et **TargetEncoder** spécifiquement pour la variable **'Surname'.**



Ce traitement et la technique StratifiedKFold seront maintenus identiques et cohérents dans les étapes suivantes pour tous les modèles réalisés.

3. Modélisation

3. a. XGB

i. XGB - Baseline

XGB - Baseline

```python
# Définir une grille de paramètres pour trouver les paramètres optimaux pour XGBoost
param_grid = {
    'classifier__n_estimators': [100, 150, 200],     # nombre d'arbres
    'classifier__max_depth': [3, 5, 7],              # profondeur des arbres
    'classifier__learning_rate': [0.01, 0.05, 0.1],  # taux d'apprentissage
    'classifier__subsample': [0.8, 1.0],             # taux d'échantillonnage pour chaque arbre
    'classifier__colsample_bytree': [0.8, 1.0],      # taux d'échantillonnage par arbre basé sur les colonnes
}

# Utiliser GridSearchCV pour trouver les meilleurs paramètres
grid_search = GridSearchCV(estimator=pipeline, param_grid=param_grid, cv=5, scoring='roc_auc', n_jobs=-1, verbose=1)

# Exécuter GridSearchCV pour trouver les meilleurs paramètres
grid_search.fit(X, y)
```

```
# Fitting 5 folds for each of 108 candidates, totalling 540 fits
# Best Parameters: {'classifier__colsample_bytree': 0.8,
#                   'classifier__learning_rate': 0.05,
#                   'classifier__max_depth': 3,
#                   'classifier__n_estimators': 200,
#                   'classifier__subsample': 1.0}
# Best ROC AUC: 0.937313804828195
```

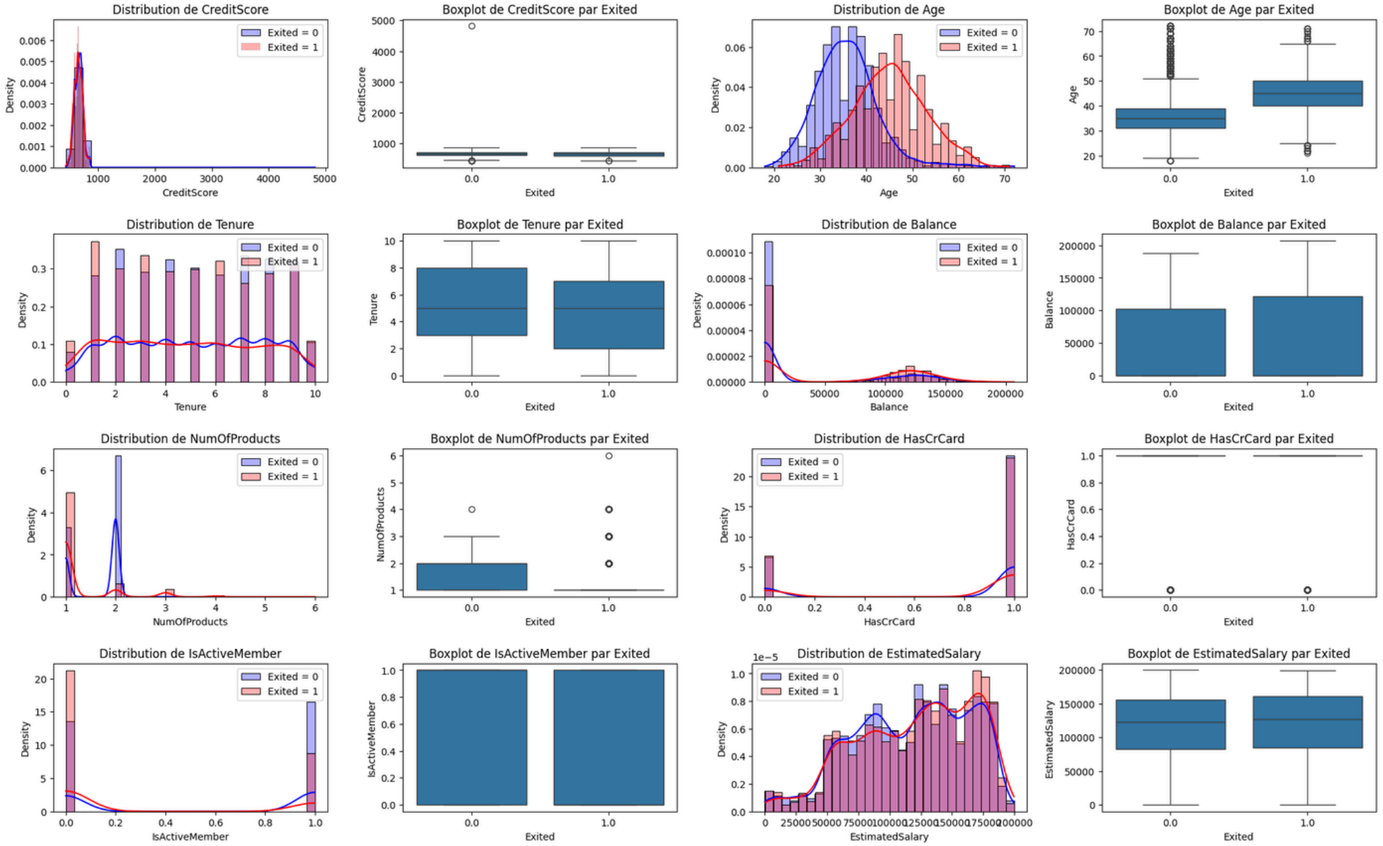
Pour commencer avec le modèle XGB, nous utilisons GridSearchCV combiné avec stratified 5 folds pour identifier un ensemble d'hyperparameters **suffisamment performant** et **stable**.

3. Modélisation

3. a. XGB

i. XGB - Baseline

ii. XGB - Feature Engineering




```python
# Liste des nouvelles colonnes numériques et catégoriques
new_numerical_features = [
    'AgeSquared', 'LogAge', 'Age_Balance_Interaction', 'LogBalance',
    'Active_Balance_Interaction', 'Product_Balance_Interaction',
    'Product_Active_Interaction', 'Age_Active_Interaction', 'FinancialStrength', 'HighRiskGroup',
    'BalanceToSalaryRatio', 'AverageTenurePerProduct'
]

new_categorical_features = [
    'AgeGroup', 'ProductGroup'
]
```

Comparaison des distributions entre 2 groupes et Feature Engineering

En analysant les graphiques comparant les deux groupes, nous pouvons identifier quelques suggestions pour le feature engineering. Elles visent à **maximiser la capacité des modèles d'apprentissage** à différencier efficacement les deux groupes Exited = 0 et Exited = 1.

À partir des graphiques, nous pouvons nous concentrer sur les groupes de certaines features :

- Groupement par **âge**
- Groupement par **Balance** (solde du compte)
- Groupement par **NumOfProducts** (nombre de produits)
- Interactions avec **IsActiveMember**
- Création de caractéristiques **combinées avancées**

Feature Engineering

```
==================================================
Évaluation de la feature: AgeSquared
AUC Train (Baseline: 0.9532): 0.9535 ± 0.0006
AUC Validation (Baseline: 0.9357): 0.9358 ± 0.0030
--------------------------------------------------
Évaluation de la feature: LogAge
AUC Train (Baseline: 0.9532): 0.9535 ± 0.0006
AUC Validation (Baseline: 0.9357): 0.9358 ± 0.0030
--------------------------------------------------
Évaluation de la feature: Age_Balance_Interaction
AUC Train (Baseline: 0.9532): 0.9535 ± 0.0006
AUC Validation (Baseline: 0.9357): 0.9358 ± 0.0029
--------------------------------------------------
Évaluation de la feature: LogBalance
AUC Train (Baseline: 0.9532): 0.9534 ± 0.0007
AUC Validation (Baseline: 0.9357): 0.9356 ± 0.0028
--------------------------------------------------
Évaluation de la feature: Active_Balance_Interaction
AUC Train (Baseline: 0.9532): 0.9533 ± 0.0006
AUC Validation (Baseline: 0.9357): 0.9354 ± 0.0026
--------------------------------------------------
Évaluation de la feature: Product_Balance_Interaction
AUC Train (Baseline: 0.9532): 0.9536 ± 0.0006
AUC Validation (Baseline: 0.9357): 0.9360 ± 0.0026
--------------------------------------------------
Évaluation de la feature: Product_Active_Interaction
AUC Train (Baseline: 0.9532): 0.9531 ± 0.0007
AUC Validation (Baseline: 0.9357): 0.9356 ± 0.0028
--------------------------------------------------
Évaluation de la feature: Age_Active_Interaction
AUC Train (Baseline: 0.9532): 0.9535 ± 0.0006
AUC Validation (Baseline: 0.9357): 0.9359 ± 0.0030
--------------------------------------------------
Évaluation de la feature: FinancialStrength
AUC Train (Baseline: 0.9532): 0.9533 ± 0.0005
AUC Validation (Baseline: 0.9357): 0.9352 ± 0.0028
--------------------------------------------------
Évaluation de la feature: HighRiskGroup
AUC Train (Baseline: 0.9532): 0.9532 ± 0.0006
AUC Validation (Baseline: 0.9357): 0.9355 ± 0.0029
```

Pour évaluer l'efficacité des nouvelles variables créées, il est nécessaire de construire un **modèle baseline** et de comparer ses performances avec celles d'un modèle intégrant les nouvelles variables.

Après avoir analysé les résultats, nous avons décidé de ne **conserver que 4 nouvelles features** parmi toutes celles créées.

À l'issue de cette étape, nous disposons d'un ensemble de hyperparameters de base ainsi que de 4 nouvelles features.

Lors de l'étape suivante, on va évaluer les performances globales du modèle actuel et procéder à un tuning.

```python
# Créer des nouvelles caractéristiques à partir des données
df['AgeGroup'] = pd.cut(df['Age'], bins=[0, 30, 50, 100], labels=['Young', 'Middle-aged', 'Senior'])
df['HasNoBalance'] = (df['Balance'] == 0).astype(int)
df['BalanceToSalaryRatio'] = df['Balance'] / (df['EstimatedSalary'] + 1e-5)
df['AverageTenurePerProduct'] = df['Tenure'] / (df['NumOfProducts'] + 1e-5)

test['AgeGroup'] = pd.cut(test['Age'], bins=[0, 30, 50, 100], labels=['Young', 'Middle-aged', 'Senior'])
test['HasNoBalance'] = (test['Balance'] == 0).astype(int)
test['BalanceToSalaryRatio'] = test['Balance'] / (test['EstimatedSalary'] + 1e-5)
test['AverageTenurePerProduct'] = test['Tenure'] / (test['NumOfProducts'] + 1e-5)
```

3. Modélisation

3. a. XGB

i. XGB - Baseline
ii. XGB - Feature Engineering
iii. XGB - Tuning

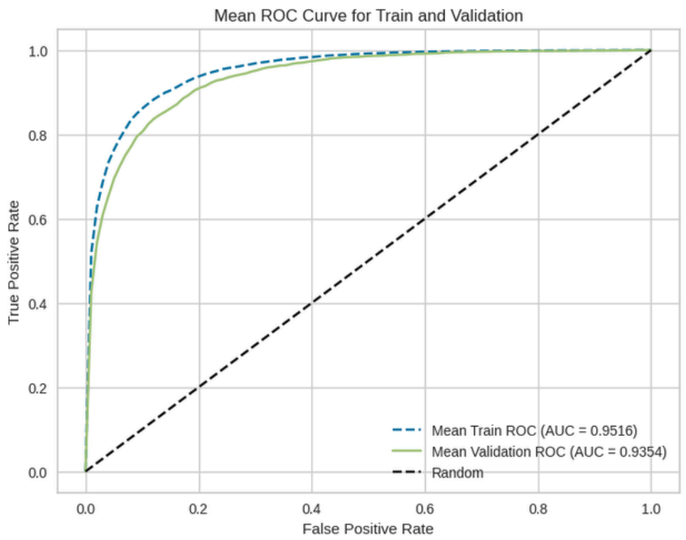
Evaluation les performances du modèle XGB actuel

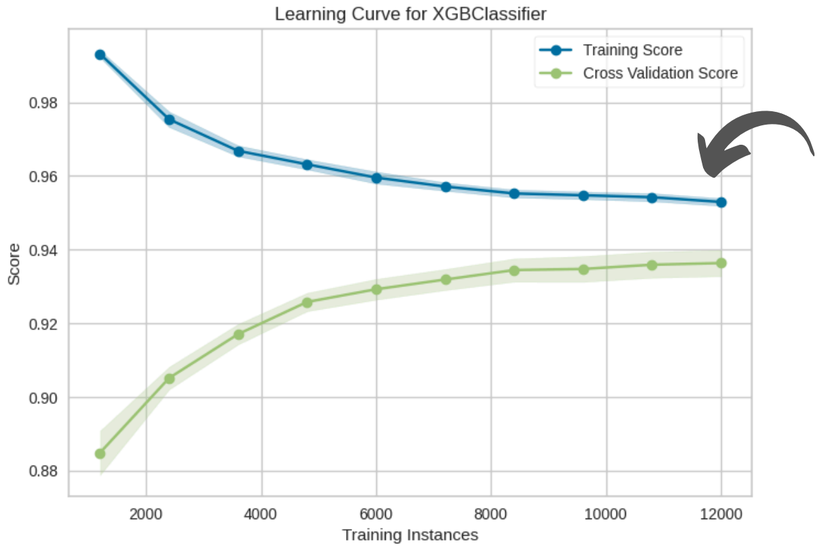
Courbe ROC moyenne (Train et Validation) :
Le modèle affiche un excellent score ROC AUC moyen : 0.9516 pour l'entraînement et 0.9354 pour la validation. La proximité entre ces deux scores indique que le modèle généralise bien sans surapprentissage significatif.

Résultats de validation croisée (par folds) :
Les scores de validation croisée varient légèrement entre 0.9328 et 0.9404, confirmant une stabilité et une robustesse du modèle sur différents sous-ensembles de données.

Courbe d'apprentissage :
Le score de validation croît progressivement avec le nombre d'instances d'entraînement et converge vers le score d'entraînement. Cela montre que le modèle bénéficierait de davantage de données pour atteindre son plein potentiel.

```
Fold 1
Train ROC AUC: 0.9523, Validation ROC AUC: 0.9389
Fold 2
Train ROC AUC: 0.9523, Validation ROC AUC: 0.9404
Fold 3
Train ROC AUC: 0.9532, Validation ROC AUC: 0.9347
Fold 4
Train ROC AUC: 0.9540, Validation ROC AUC: 0.9328
Fold 5
Train ROC AUC: 0.9538, Validation ROC AUC: 0.9356
```





Résultats SHAP

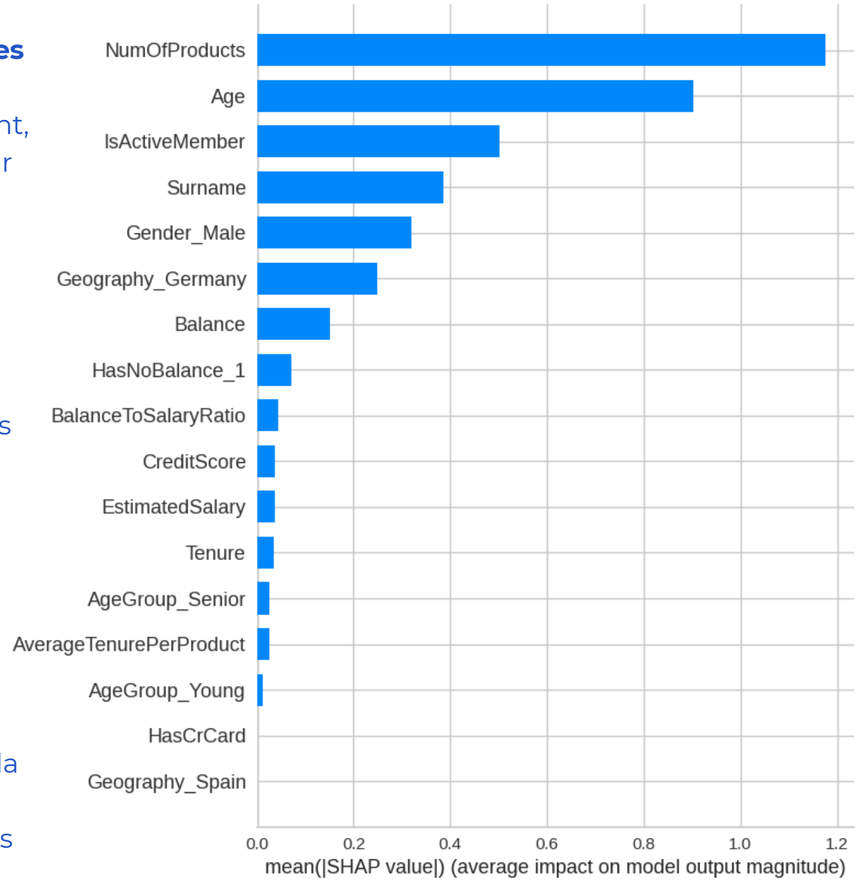
Les résultats SHAP mettent en évidence l'importance des comportements liés aux produits et à l'âge pour expliquer le churn. **Si le modèle se concentre trop sur certaines features principales tout en négligeant les autres, il risque de devenir plus sensible au bruit dans ces features.** Cependant, dans ce cas précis, cela ne pose pas réellement problème, car ces features ont également été identifiées comme importantes lors des analyses précédentes.

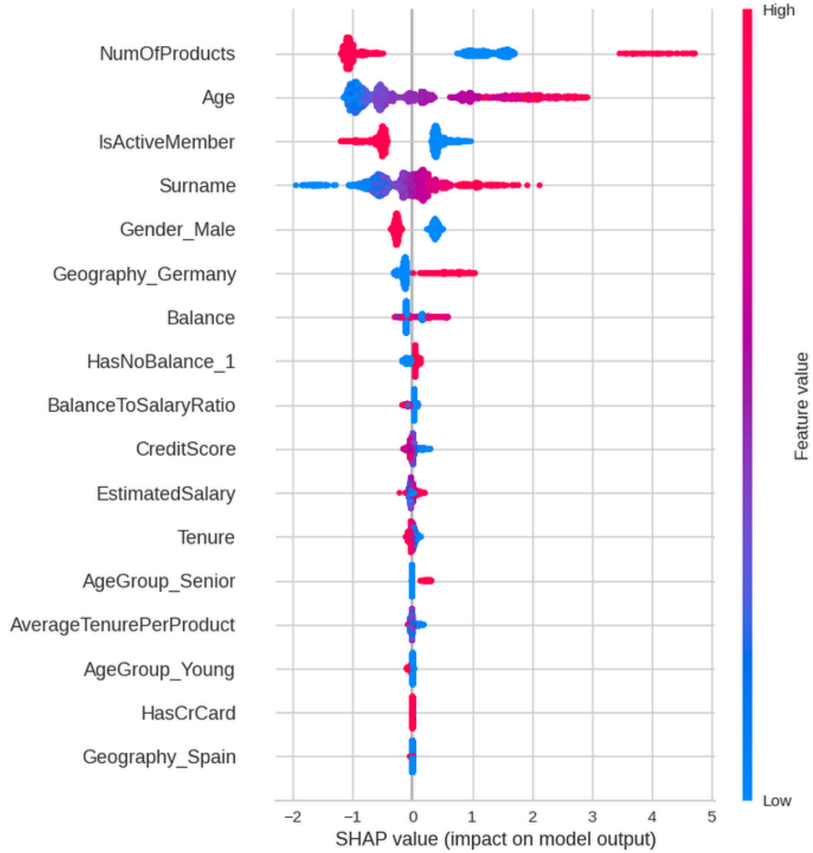
Graphique d'importance moyenne des features :

- Les variables les plus influentes sur les prédictions sont **NumOfProducts, Age, et IsActiveMember**. Elles contribuent significativement à la séparation des groupes (Exited = 0 et Exited = 1).
- Les variables comme **Balance, CreditScore, et EstimatedSalary** ont un impact relativement faible, indiquant qu'elles sont moins déterminantes pour le modèle.

Graphique de distribution des valeurs SHAP :

- Les valeurs élevées de **NumOfProducts et Age** augmentent fortement la probabilité qu'un client quitte la banque.
- **IsActiveMember** montre une relation inversée : les clients moins actifs sont plus susceptibles de churn.
- Les interactions complexes entre les variables comme **Gender_Male** et **Geography_Germany** sont visibles, mais leur impact est moindre comparé aux trois principales features.





XGB - Tuning

Après avoir analysé les différents aspects, nous constatons que le modèle XGB a presque atteint son plein potentiel. Cependant, nous pouvons encore essayer quelques étapes de tuning supplémentaires pour éventuellement trouver une meilleure combinaison de paramètres, si elle existe.

On utilise de **Hyperopt** avec **l'algorithme Tree-structured Parzen Estimator (TPE)** pour explorer l'espace des hyperparamètres. Il s'agit d'**une méthode d'optimisation bayésienne** qui permet de rechercher efficacement les meilleures valeurs possibles.

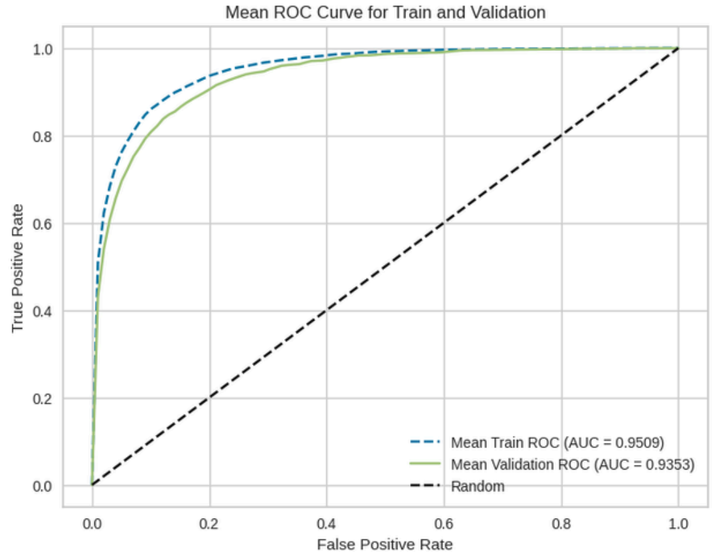
```
space = {
    'n_estimators': hp.quniform('n_estimators', 150, 300, 5),
    'max_depth': hp.quniform('max_depth', 2, 5, 1),
    'learning_rate': hp.uniform('learning_rate', 0.03, 0.07),
    'subsample': hp.uniform('subsample', 0.8, 1.0),
    'colsample_bytree': hp.uniform('colsample_bytree', 0.7, 0.9),
    'min_child_weight': hp.quniform('min_child_weight', 1, 3, 1),
    'gamma': hp.uniform('gamma', 0, 1),
    'reg_alpha': hp.uniform('reg_alpha', 0, 1),
    'reg_lambda': hp.uniform('reg_lambda', 0, 1),
    'scale_pos_weight': hp.quniform('scale_pos_weight', 1, 3, 1)
}
```

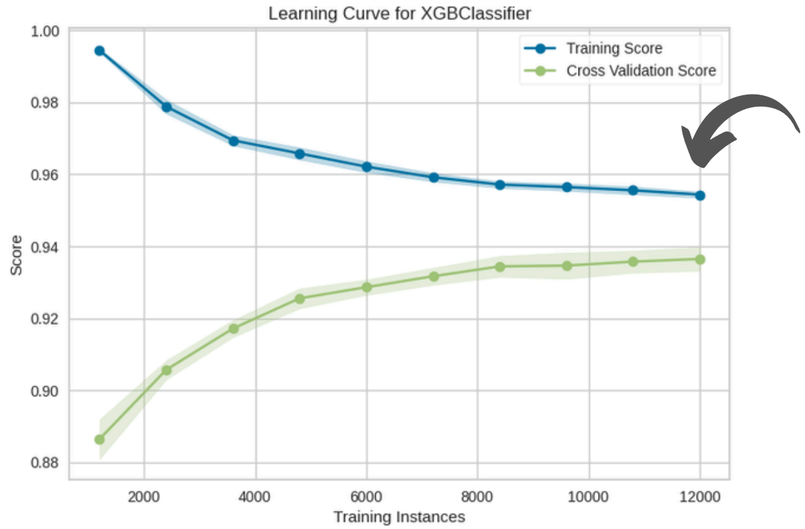
```
# Best hyperparameters: {'colsample_bytree': 0.7978358403494772,
        # 'gamma': 0.47954324947744753,
        # 'learning_rate': 0.04225027097955838,
        # 'max_depth': 3.0,
        # 'min_child_weight': 1.0,
        # 'n_estimators': 290.0,
        # 'reg_alpha': 0.3029419578747556,
        # 'reg_lambda': 0.999254418512059,
        # 'scale_pos_weight': 1.0,
        # 'subsample': 0.855781390548203 9}
# Best ROC AUC: 0.9366689965719761
# Mean Best Iteration: 274
```

XGB - Tuning

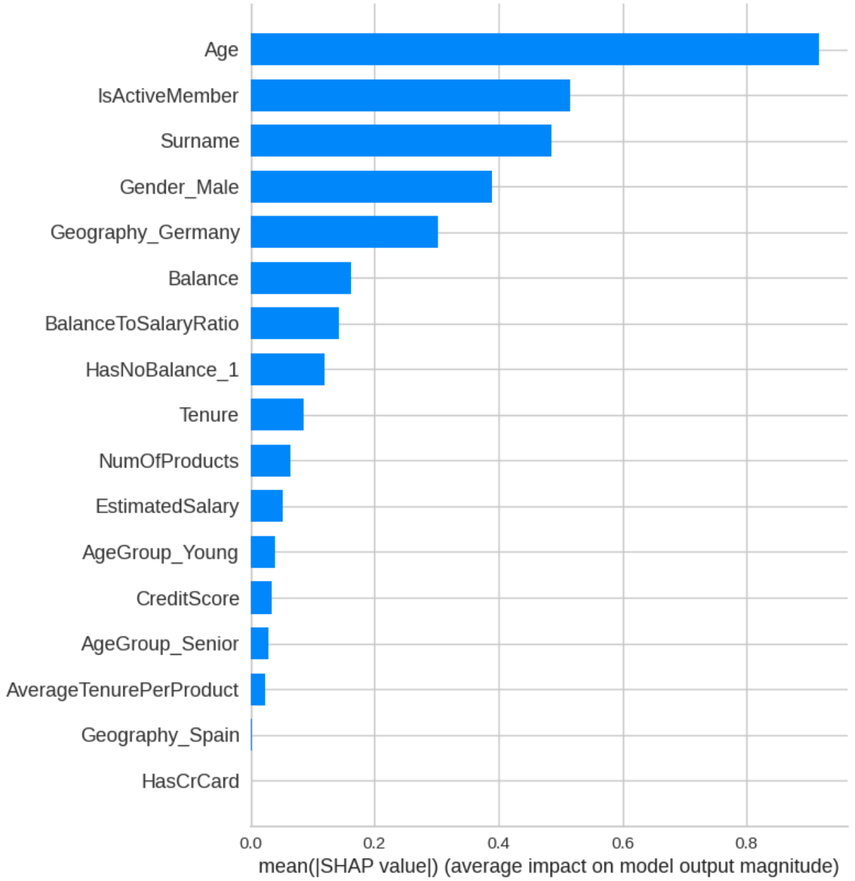
Après le tuning et l'application des meilleurs paramètres, nous constatons que les résultats n'ont pas montré d'amélioration significative par rapport aux anciens paramètres.

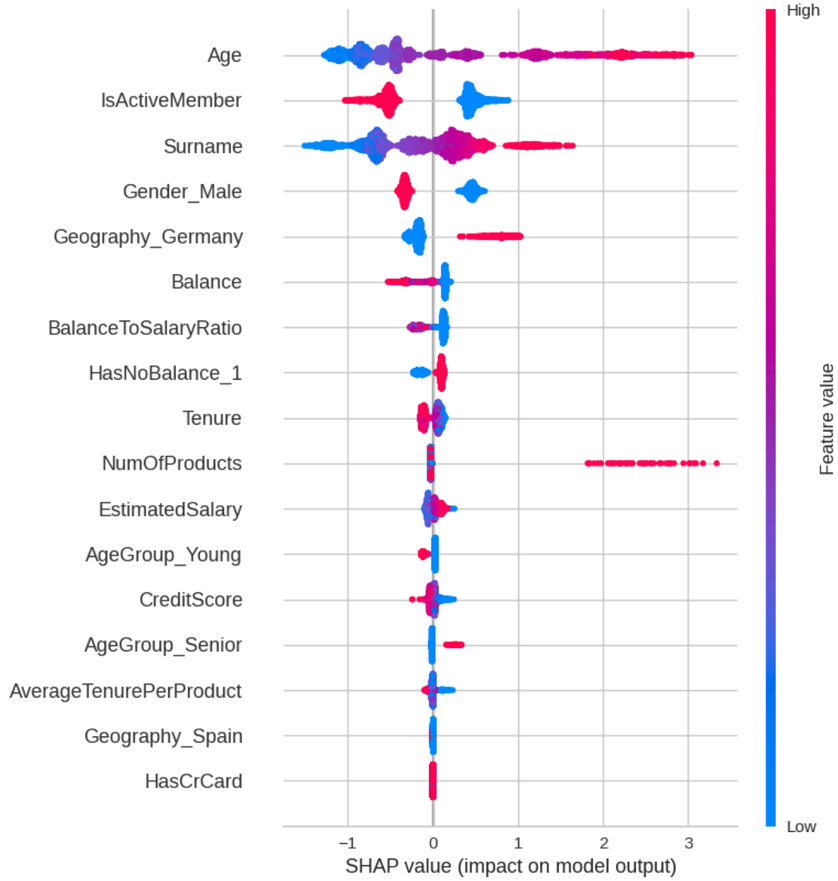
On peut donc dire que les anciens paramètres avaient déjà atteint leur plein potentielres valeurs possibles.

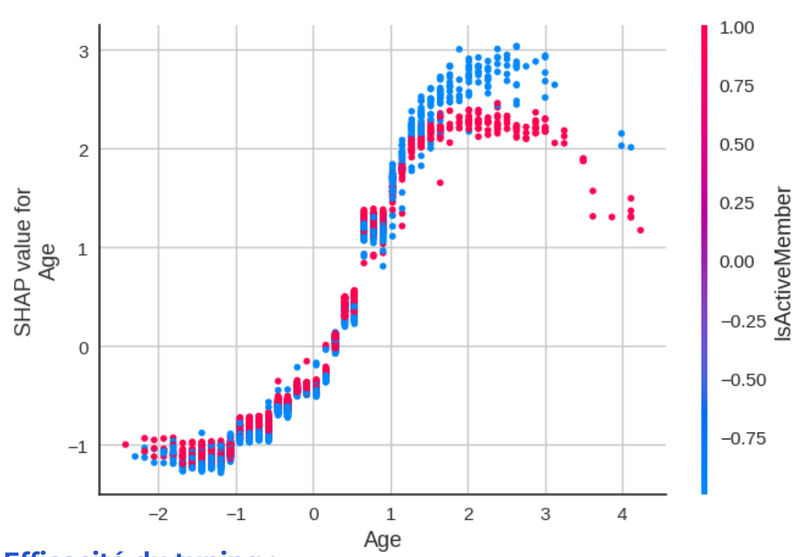




XGB - Tuning







Efficacité du tuning :

Comme vu précédemment, le tuning ne semble pas avoir apporté de changement significatif aux performances globales du modèle, car les features importantes et la distribution des valeurs SHAP restent similaires.

Cependant, **le tuning a amélioré l'utilisation de certaines features secondaires et optimisé les hyperparamètres, ce qui pourrait augmenter la stabilité et la capacité de généralisation du modèle.**

3. Modélisation

3. b. Stacking

Eléments clés pour le modèle de Stacking efficace

```
Fold 1 - XGBoost ROC AUC: 0.9384
Fold 1 - LightGBM ROC AUC: 0.9387
Fold 1 - CatBoost ROC AUC: 0.9405
Fold 1 - RandomForest ROC AUC: 0.9343
Fold 1 - Logistic Regression ROC AUC: 0.8925
Fold 1 - MLPClassifier ROC AUC: 0.9321
------------------------------------------------
Fold 2 - XGBoost ROC AUC: 0.9392
Fold 2 - LightGBM ROC AUC: 0.9397
Fold 2 - CatBoost ROC AUC: 0.9460
Fold 2 - RandomForest ROC AUC: 0.9375
Fold 2 - Logistic Regression ROC AUC: 0.9005
Fold 2 - MLPClassifier ROC AUC: 0.9385
------------------------------------------------
Fold 3 - XGBoost ROC AUC: 0.9344
Fold 3 - LightGBM ROC AUC: 0.9347
Fold 3 - CatBoost ROC AUC: 0.9352
Fold 3 - RandomForest ROC AUC: 0.9287
Fold 3 - Logistic Regression ROC AUC: 0.8950
Fold 3 - MLPClassifier ROC AUC: 0.9255
------------------------------------------------
Fold 4 - XGBoost ROC AUC: 0.9324
Fold 4 - LightGBM ROC AUC: 0.9320
Fold 4 - CatBoost ROC AUC: 0.9347
Fold 4 - RandomForest ROC AUC: 0.9297
Fold 4 - Logistic Regression ROC AUC: 0.8858
Fold 4 - MLPClassifier ROC AUC: 0.9270
------------------------------------------------
Fold 5 - XGBoost ROC AUC: 0.9349
Fold 5 - LightGBM ROC AUC: 0.9347
Fold 5 - CatBoost ROC AUC: 0.9336
Fold 5 - RandomForest ROC AUC: 0.9308
Fold 5 - Logistic Regression ROC AUC: 0.8829
Fold 5 - MLPClassifier ROC AUC: 0.9223
------------------------------------------------
```

Pour utiliser efficacement le modèle de stacking, voici les éléments clés :

1. Diversité entre les modèles de base (Base Models)

- L'efficacité du Stacking dépend de la capacité des modèles de base à capturer différents aspects des données. Plus les modèles sont diversifiés, meilleure est la performance du modèle final.

2. Modèle méta (Meta-Model)

- Le modèle méta est le dernier modèle qui combine les prédictions des modèles de base. Ce modèle doit être **simple** (par ex. : une régression logistique) **pour éviter l'overfitting**.
- Un modèle méta bien **optimisé** peut exploiter efficacement les relations entre les prédictions des modèles de base.

3. Qualité des modèles de base

- Les modèles de base doivent être **suffisamment performants** pour fournir des informations utiles au modèle méta. Si les modèles de base sont trop faibles, le modèle méta ne pourra pas améliorer significativement les résultats.
- Cependant, **il est important que les modèles de base soient complémentaires** plutôt que tous très puissants.

Par conséquent, dans notre cas, on utilise une diversité de **6 modèles différents** et **6 preprocessors distincts** afin d'exploiter au maximum la diversité de chaque modèle.

Meta Model - Logistic Regression

```
Fold 1 - Stacking Train ROC AUC: 0.9380, Test ROC AUC: 0.9413
Fold 2 - Stacking Train ROC AUC: 0.9371, Test ROC AUC: 0.9456
Fold 3 - Stacking Train ROC AUC: 0.9395, Test ROC AUC: 0.9356
Fold 4 - Stacking Train ROC AUC: 0.9395, Test ROC AUC: 0.9353
Fold 5 - Stacking Train ROC AUC: 0.9396, Test ROC AUC: 0.9354
================================================
Mean Stacking Train ROC AUC: 0.9387
Mean Stacking Test ROC AUC: 0.9386
================================================
Les prédictions ont été enregistrées dans 'test_predictions_stacked_stk_v9.csv'.
```

*Résultat par un meta-model **LogisticRegression** baseline (il n'équilibre pas les poids des modèles à ce stade)*

```python
# 1. Ajouter GridSearchCV pour optimiser LogisticRegression en tant que méta-classifier
param_grid = {
    'C': [0.01, 0.1, 1, 10],
    'penalty': ['l1', 'l2'],
    'solver': ['liblinear'],  # Prise en charge des pénalités l1 et l2
}
grid_search = GridSearchCV(
    LogisticRegression(max_iter=1000, random_state=42),
    param_grid,
    cv=5,
    scoring='roc_auc',
    verbose=1
)

# Exécuter la recherche de grille pour ajuster les hyperparamètres
grid_search.fit(X_meta, y)

# Obtenir le meilleur modèle à partir de GridSearchCV
meta_classifier = grid_search.best_estimator_

print("Best hyperparameters for Logistic Regression (meta-classifier):")
print(grid_search.best_params_)
```

- Pour éviter l'overfitting, nous utilisons un **meta modèle simple** comme **LogisticRegression** en tant que base afin d'observer les résultats baseline.
- Ensuite, à partir de ce modèle baseline, nous pouvons continuer à **optimiser les hyperparamètres pour le meta-modèle.**
- Dans ce cas, nous avons **bien optimisé la recherche des meilleurs paramètres avec un GridSearchCV**.
- De plus, on a bien **ajusté les poids des modèles en fonction de leurs performances**.

Malheureusement, le résultat reste identique et n'améliore pas notre ranking.

```
Fitting 5 folds for each of 8 candidates, totalling 40 fits
Best hyperparameters for Logistic Regression (meta-classifier):
{'C': 1, 'penalty': 'l2', 'solver': 'liblinear'}
Weights for stacking based on mean AUCs:
XGBoost: 0.1682
LightGBM: 0.1683
CatBoost: 0.1686
RandomForest: 0.1676
LogisticRegression: 0.1602
MLP: 0.1670
```

```
Fold 1 - Stacking Train ROC AUC: 0.9380, Test ROC AUC: 0.9412
Fold 2 - Stacking Train ROC AUC: 0.9371, Test ROC AUC: 0.9456
Fold 3 - Stacking Train ROC AUC: 0.9395, Test ROC AUC: 0.9355
Fold 4 - Stacking Train ROC AUC: 0.9395, Test ROC AUC: 0.9352
Fold 5 - Stacking Train ROC AUC: 0.9395, Test ROC AUC: 0.9354
================================================
Mean Stacking Train ROC AUC: 0.9387
Mean Stacking Test ROC AUC: 0.9386
================================================
Les prédictions ont été enregistrées dans 'test_predictions_stacked_stk_v9_weighted_GridSearchCV.csv'.
```

*Résultat par un meta-model **LogisticRegression** après l'utilisation d'un **GridSearchCV** et les **poids des modèles pondérés***

Meta Model - GradientBoostingClassifier

```
Fold 1 - Stacking Train ROC AUC: 0.9401, Test ROC AUC: 0.9409
Fold 2 - Stacking Train ROC AUC: 0.9391, Test ROC AUC: 0.9452
Fold 3 - Stacking Train ROC AUC: 0.9415, Test ROC AUC: 0.9352
Fold 4 - Stacking Train ROC AUC: 0.9416, Test ROC AUC: 0.9344
Fold 5 - Stacking Train ROC AUC: 0.9415, Test ROC AUC: 0.9352
==================================================
Mean Stacking Train ROC AUC: 0.9408
Mean Stacking Test ROC AUC: 0.9382
==================================================
Les prédictions ont été enregistrées dans 'test_predictions_stacked_stk_v9_weighted_optuna_GradientBoost
```

*Résultat par un Meta-Model **GradientBoostingClassifier**, avec les **poids des modèles inchangés,** hérités exactement de l'étape précédente.*

```python
# Fonction d'objectif pour Optuna
def objective(trial):
    # Définir les hyperparamètres à optimiser
    n_estimators = trial.suggest_int("n_estimators", 50, 150)  # Nombre d'arbres
    learning_rate = trial.suggest_loguniform("learning_rate", 0.01, 0.1)  # Taux d'apprentissage
    max_depth = trial.suggest_int("max_depth", 2, 3)  # Profondeur maximale
    min_samples_split = trial.suggest_int("min_samples_split", 10, 20)  # Minimum pour diviser un nœud
    min_samples_leaf = trial.suggest_int("min_samples_leaf", 5, 10)  # Minimum d'échantillons dans une feuille
    subsample = trial.suggest_float("subsample", 0.8, 1.0)  # Fraction des échantillons utilisés par arbre

    # Initialiser le modèle GradientBoostingClassifier avec les paramètres suggérés
    meta_classifier = GradientBoostingClassifier(
        n_estimators=n_estimators,
        learning_rate=learning_rate,
        max_depth=max_depth,
        min_samples_split=min_samples_split,
        min_samples_leaf=min_samples_leaf,
        subsample=subsample,
        random_state=42
    )

    # Initialiser la validation croisée
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = []
```

Quand faut-il utiliser un meta-modèle complexe ?

- Lorsque les **modèles de base ont des relations complexes *(notre cas)*** : Si les modèles de base sont hétérogènes (heterogeneous) et produisent des prédictions très différentes, un meta-modèle complexe comme GradientBoostingClassifier, RandomForest, ou un Neural Network peut apprendre des relations non linéaires entre les sorties.
- Pour des données **volumineuses et multidimensionnelles** : Si le nombre de modèles de base est élevé ou si chaque modèle génère plusieurs caractéristiques de sortie (par exemple, probability scores, logits, etc.), un meta-modèle complexe peut mieux exploiter ces informations.
- Lorsque les **modèles de base n'atteignent pas une performance optimale *(notre cas)*** : Si les modèles de base ne sont pas suffisamment performants, un meta-modèle complexe peut compenser leurs faiblesses et améliorer les résultats globaux.

De manière similaire au traitement des modèles de base, pour **éviter l'overfitting**, des **techniques de cross-validation** ainsi que des **techniques de régularisation** soigneusement appliquées sur les hyperparamètres sont **très essentielles** pour évaluer le niveau de surapprentissage.
Dans notre cas, on utilise **Optuna** (qui repose sur un algorithme appelé **Tree-structured Parzen Estimator (TPE)** et sur une approche basée sur la r**echerche bayésienne**) pour automatiser et accélérer le processus de sélection des hyperparamètres optimaux)

Les résultats sont extrêmement satisfaisants et ne montrent aucun signe d'overfitting. Ils ont été grandement améliorés sur les 20% du jeu de test sur Kaggle, ce qui nous place à la 1ère position dans le classement à ce stade.

Meta Model - XGBClassifier

```
# Meilleurs hyperparamètres : {'n_estimators': 110,
#                              'learning_rate': 0.0358646498763739,
#                              'max_depth': 2, 'min_child_weight': 1,
#                              'subsample': 0.8545558590904705,
#                              'colsample_bytree': 0.8638731010436426,
#                              'gamma': 2.185987853832697,
#                              'scale_pos_weight': 5.879010035333382}
# Meilleur AUC : 0.9387268552683242
```

```
Fold 1 - Stacking Train ROC AUC: 0.9405, Test ROC AUC: 0.9412
Fold 2 - Stacking Train ROC AUC: 0.9395, Test ROC AUC: 0.9451
Fold 3 - Stacking Train ROC AUC: 0.9417, Test ROC AUC: 0.9359
Fold 4 - Stacking Train ROC AUC: 0.9419, Test ROC AUC: 0.9353
Fold 5 - Stacking Train ROC AUC: 0.9417, Test ROC AUC: 0.9358
================================================
Mean Stacking Train ROC AUC: 0.9411
Mean Stacking Test ROC AUC: 0.9387
================================================
Les prédictions ont été enregistrées dans 'test_predictions_stacked_stk_v9_weighted_optuna_XGB.csv'.
```

*Résultat par un Meta-Model **XGBClassifier**, avec les **poids des modèles inchangés,** hérités exactement de l'étape précédente.*

```python
# Fonction d'objectif pour Optuna
def objective(trial):
    # Définir les hyperparamètres à optimiser pour XGBoost
    n_estimators = trial.suggest_int("n_estimators", 50, 150)  # Nombre d'arbres
    learning_rate = trial.suggest_loguniform("learning_rate", 0.01, 0.1)  # Taux
    max_depth = trial.suggest_int("max_depth", 2, 10)  # Profondeur maximale
    min_child_weight = trial.suggest_int("min_child_weight", 1, 10)  # Poids min
    subsample = trial.suggest_float("subsample", 0.8, 1.0)  # Fraction des échan
    colsample_bytree = trial.suggest_float("colsample_bytree", 0.5, 1.0)  # Frac
    gamma = trial.suggest_float("gamma", 0, 5)  # Régularisation pour le gain de
    scale_pos_weight = trial.suggest_float("scale_pos_weight", 1, 10)  # Balance

    # Initialiser le modèle XGBClassifier avec les paramètres suggérés
    meta_classifier = xgb.XGBClassifier(
        n_estimators=n_estimators,
        learning_rate=learning_rate,
        max_depth=max_depth,
        min_child_weight=min_child_weight,
        subsample=subsample,
        colsample_bytree=colsample_bytree,
        gamma=gamma,
        scale_pos_weight=scale_pos_weight,
        random_state=42,
        use_label_encoder=False,  # Désactiver l'avertissement
        eval_metric="auc"  # Mesure de performance utilisée
    )

    # Initialiser la validation croisée
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = []
```

En reprenant exactement la même idée que pour le GradientBoostingClassifier, nous testons un **meta-modèle XGBClassifier** avec **Optuna** pour chercher ses meilleurs hyperparamètres.
Les résultats sont également **améliorés et ne présentent pas d'overfitting**, avec une progression notable sur les 20 % du jeu de test (C'est le meilleur résultat que nous avons obtenu avec un score de **0,9371** sur Kaggle). Ils sont presque similaires entre l'ensemble de validation et les 20 % de l'ensemble de test.

3. Modélisation

3. c. Voting

Modèle de Voting

```python
# Créer une instance de VotingClassifier
voting_clf = VotingClassifier(
    estimators=[
        ('xgb', xgb_classifier),
        ('lgb', lgb_classifier),
        ('cat', cat_classifier),
        ('rf', rf_classifier),
        ('mlp', mlp_classifier),
        ('lr', lr_classifier)
    ],
    voting='soft',  # Utiliser la moyenne pondérée des probabilités
    n_jobs=-1
)

# Création du pipeline
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),  # Étape de prétraitement
    ('voting_clf', voting_clf)       # Étape du VotingClassifier
])

# Fonction objectif pour Optuna
def objective(trial):
    # Définir les poids pour chaque modèle du VotingClassifier
    weights = trial.suggest_float('xgb_weight', 0.0, 1.0), \
              trial.suggest_float('lgb_weight', 0.0, 1.0), \
              trial.suggest_float('cat_weight', 0.0, 1.0), \
              trial.suggest_float('rf_weight', 0.0, 1.0), \
              trial.suggest_float('mlp_weight', 0.0, 1.0,), \
              trial.suggest_float('lr_weight', 0.0, 0.5)

    # Normaliser les poids pour s'assurer que leur somme = 1
    total = sum(weights)
    normalized_weights = [w / total for w in weights]
```

La méthode Voting est aussi une technique d'ensemble (ensemble learning) comme le stacking, utilisée pour combiner les prédictions de plusieurs modèles afin d'améliorer la performance globale avec 2 types de voting.

1. **Hard Voting :**
   - Les prédictions finales sont basées sur la **majorité** des votes des modèles.
   - Exemple : Si trois modèles prédisent les classes suivantes pour une observation donnée :
     - Modèle 1 : Classe A
     - Modèle 2 : Classe B
     - Modèle 3 : Classe A La classe A est choisie comme prédiction finale.
2. **Soft Voting :**
   - Les prédictions sont basées sur les **probabilités moyennes** pour chaque classe.
   - Exemple : Si les modèles donnent les probabilités suivantes pour la classe A :
     - Modèle 1 : 0,7
     - Modèle 2 : 0,6
     - Modèle 3 : 0,8 La moyenne est 0,7, et la classe A est prédite.

Comparaison entre 2 techniques Voting et Stacking :

- **Voting** : Si certains modèles de base sont faibles ou bruités, ils peuvent affecter négativement le résultat final, car **Voting ne corrige pas ces faiblesses**.
- **Stacking** : Le meta-modèle dans Stacking **peut compenser les faiblesses des modèles de base** en attribuant moins de poids à leurs prédictions ou en apprenant à mieux combiner leurs sorties.

Modèle de Voting

```
Meilleurs poids : {'xgb_weight': 0.5859285378045953, 'lgb_weight': 0.4691216224725798, 'cat_weight': 0.8852464595669012, 'rf_weight': 0.07572771426854208, 'mlp_weight': 0.518964
Meilleur ROC AUC : 0.9365880996605391
Fold 1:
Train ROC AUC: 0.9551
Validation ROC AUC: 0.9389
--------------------------------------------------
Fold 2:
Train ROC AUC: 0.9543
Validation ROC AUC: 0.9412
--------------------------------------------------
Fold 3:
Train ROC AUC: 0.9566
Validation ROC AUC: 0.9348
--------------------------------------------------
Fold 4:
Train ROC AUC: 0.9562
Validation ROC AUC: 0.9328
--------------------------------------------------
Fold 5:
Train ROC AUC: 0.9574
Validation ROC AUC: 0.9352
--------------------------------------------------
==================================================
Mean Train ROC AUC: 0.9559
Mean Validation ROC AUC: 0.9366
==================================================
Les prédictions ont été enregistrées dans 'test_predictions_voting_optuna.csv'.
```

Dans notre cas, nous avons observé un phénomène d'**overfitting assez significati**f avec la méthode de voting. Nous avons utilisé **Optuna** pour déterminer les poids des modèles afin d'améliorer les performances.

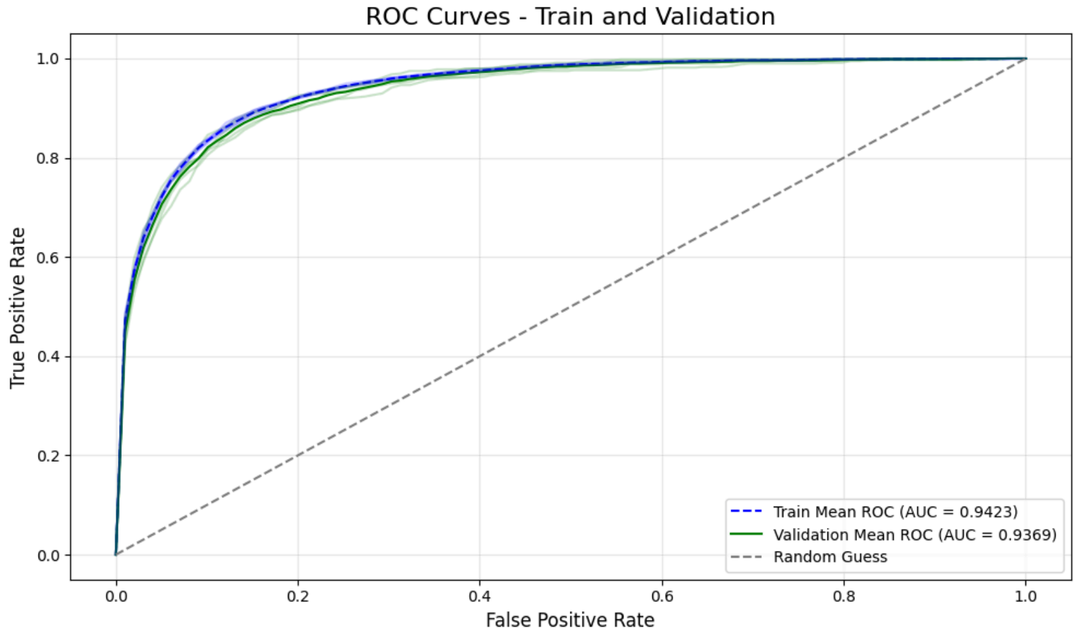
Les résultats du Voting sont moins bons que ceux du Stacking et montrent un léger overfitting.

3. Modélisation

3. d. Catboost

Meta Model - Catboost

```
Fold 1
Fold 1 - CatBoost Train ROC AUC: 0.9424, Validation ROC AUC: 0.9406
Fold 2
Fold 2 - CatBoost Train ROC AUC: 0.9429, Validation ROC AUC: 0.9463
Fold 3
Fold 3 - CatBoost Train ROC AUC: 0.9442, Validation ROC AUC: 0.9354
Fold 4
Fold 4 - CatBoost Train ROC AUC: 0.9445, Validation ROC AUC: 0.9348
Fold 5
Fold 5 - CatBoost Train ROC AUC: 0.9440, Validation ROC AUC: 0.9336
================================================
Mean CatBoost Train ROC AUC: 0.9436
Mean CatBoost Validation ROC AUC: 0.9381
================================================
Les prédictions ont été enregistrées dans 'test_predictions_cb.csv'.
```



Après avoir observé les performances des modèles de base, on constate que le modèle **CatBoost dépasse systématiquement les autres en termes de performance**. Ainsi, nous pouvons le séparer pour explorer davantage son potentiel.

Le modèle CatBoost fonctionne bien sur l'ensemble de validation avec **des résultats assez proches de ceux du modèle Stacking**. Cependant, lorsque nous avons soumis ces résultats sur Kaggle, les performances ont **chuté de manière significative**.

Par conséquent, nous avons décidé de **ne pas explorer davantage** les performances de ce modèle.

MOSEF 2024-2025

12/2024

Thank you

Ce projet est le fruit d’un apprentissage collectif et de votre guidance inestimable.
Merci sincèrement pour votre patience et pour avoir partagé votre passion pour la data science avec nous.

HUYNH Minh Nhat Thy
Sara Laval-Jeantet